

COMP9517: Computer Vision

2026 T3 Lab 1 Specification

Maximum Marks Achievable: 2.5

This lab is **worth 2.5% of the total course mark.**

The lab must be submitted online.

Instructions for submission will be posted closer to the deadline.

Deadline for submission is Week 3, Friday 2 October 2026, 18:00:00 AET.

Objective: This lab revisits important concepts covered in the Week 1 and Week 2 lectures and aims to make you familiar with implementing specific algorithms.

Software: You are required to use OpenCV 3+ with Python 3+ and submit your code as a Jupyter notebook (see coding and submission requirements below). In the tutor consultation session this week, your tutors will give a demo of the software to be used, and you can ask any questions you may have about this lab.

Materials: The images to be used in this lab are available via WebCMS3 (.zip file).

Submission: All code and requested results are assessable after the lab. Submit your source code as a Jupyter notebook (.ipynb file) that includes all output and answers to all questions (see coding requirements at the end of this document) by the above deadline. The submission link will be announced in due time.

Task 1 (0.75 mark)

On a walk through Sydney, you decide to take a picture of the Opera House, but your camera is defective and produces very noisy images. You remember from the computer vision course that you can still create a nice picture simply by averaging many images, so you take 10 images from the same viewpoint (see example on the next page).

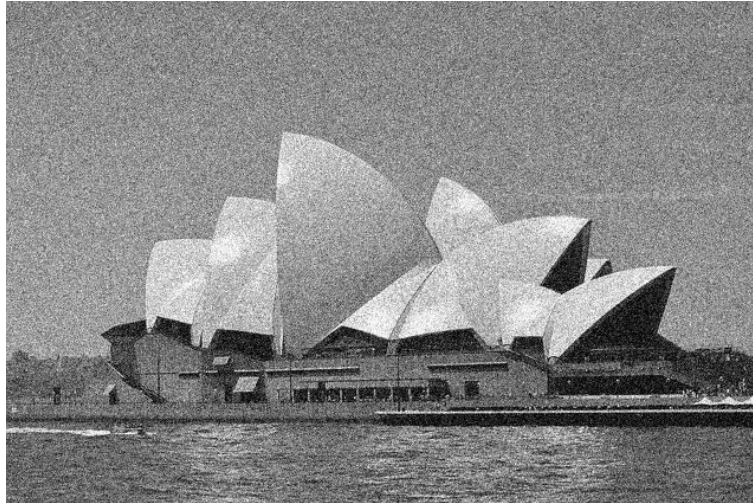
In your notebook, write an algorithm that calculates the average image of the 10 noisy images, named **Opera01.jpg–Opera10.jpg** (see the given .zip file), and displays the result.

Also answer the following questions in your notebook:

Theoretically, how much noise reduction should you be able to achieve this way? That is, by what factor should the standard deviation of the noise drop?

Practically, how much noise reduction did you actually achieve here? Note that this requires measuring the standard deviation of the pixel values within a region containing only random noise and no other image structures. The best region for this is the sky.

Measure the standard deviation within some rectangular region in the top-left part of the sky in one of the single noisy images and within the same region in the average image. Report the two standard deviations and the ratio of the former to the latter.



Task 2 (0.75 mark)

Edges are an important source of semantic information in images. They occur in human visual perception at divisions between areas of different intensity, colour, or texture. A gray-scale image can be thought of as a 2D landscape with areas of different intensity living at different heights. A transition between areas of different intensity in an image I means there must be a steep slope, which we formalise as the gradient (vector):

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right) \quad (2)$$

As the image I is discrete, we need to approximate the continuous derivatives $\partial I / \partial x$ and $\partial I / \partial y$ by finite differences. The following are simple examples of convolution kernels that perform finite differencing in x and in y , respectively:

$$P_x = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \text{ and } P_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Write an algorithm that computes the two derivative images, $\partial I / \partial x \approx I * P_x$ and $\partial I / \partial y \approx I * P_y$ from an input image. Use the average image from Task 1 to test your algorithm.

Notice that the calculations may produce negative output pixel values. Thus, make sure you use the right image data types for the calculations.

After that, extend the algorithm to compute the gradient magnitude image:

$$\|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2} \quad (3)$$

In other words, the algorithm must create a new output image having the same size as the input image and the derivative images, and then for every pixel in the output image compute the value as the square root of the sum of the squared value of the derivative image $\partial I/\partial x$ and the squared value of the derivative image $\partial I/\partial y$ at that pixel position.

Here again, notice that the calculations may produce intermediate values outside the 8-bit range, so make sure you use the right image data types for the calculations.

In your notebook, display the final output image of your algorithm, which should look like this:

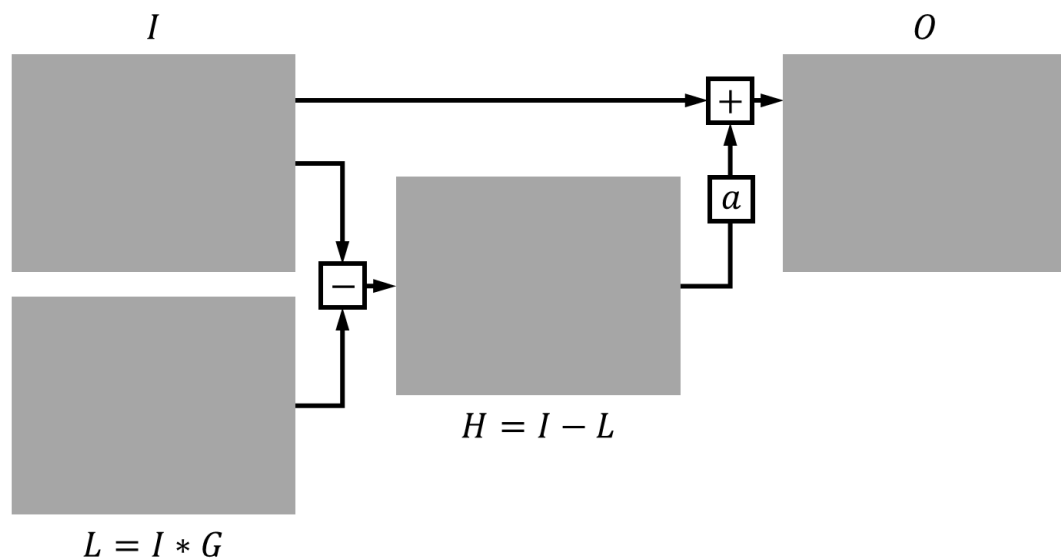


Also answer the following question in your notebook: What is the name of the convolution kernels used in this task?

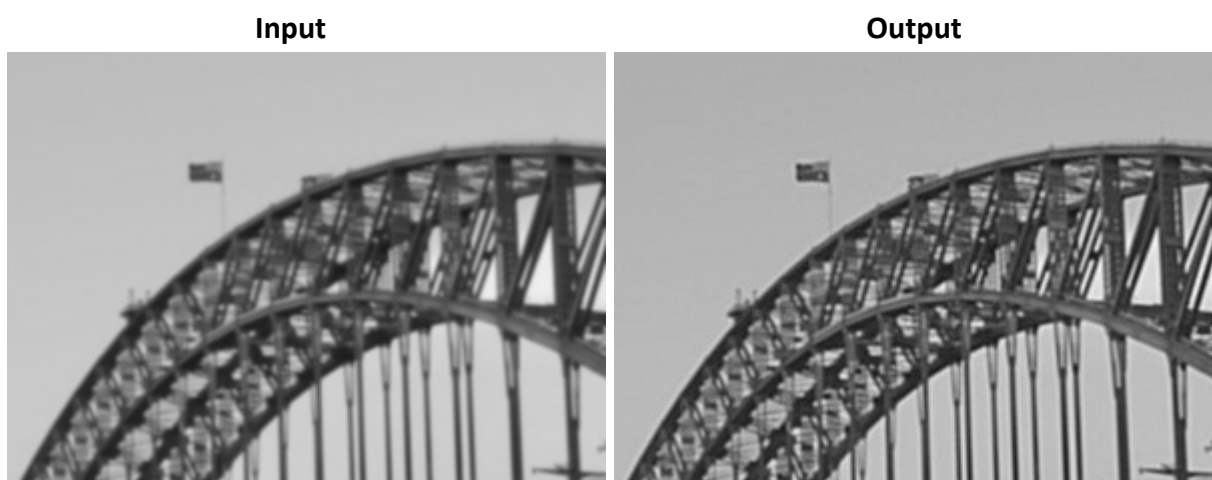
Task 3 (1 mark)

On another walk through Sydney, you take a picture of the bridge, but afterwards you notice that the image is not as sharp as it could be, and the fine details look a bit blurred. From the computer vision course, you remember that an image can be sharpened by using specific filters that enhance the high frequencies in the image.

One possible technique to accomplish this is called unsharp masking. The figure below gives a schematic overview of this technique. In words, it takes the input image I and convolves it with a Gaussian kernel G with standard deviation σ (also called the scale), resulting in a slightly blurred image L . Next, the blurred image L is subtracted pixelwise from the input image I , resulting in image H , in which the high frequencies are enhanced and the low frequencies suppressed. Then, each pixel in H is multiplied by a constant factor a , and the resulting image is added pixelwise to the input image I , resulting in output image O .



Write an algorithm that implements the above technique. Apply it to the image **Bridge.jpg** (see the provided .zip file), using $\sigma = 1.0$ pixels and $a = 2.0$, and display the result. The differences between I and O are subtle, but the latter should be visibly sharper, especially if you zoom in on the details (as seen in the example below).



Like in the previous task, this technique may produce pixel values outside the 8-bit range, so make sure you use the right image data types for the computations.

In your notebook, define the parameters σ and α as variables at the beginning of your code for this task, and then use these variables further down your code where applicable. In other words, separate parameter values from the actual algorithm, so that it is easy afterwards to change them and play around with different values.

Coding Requirements

Make sure that in your Jupyter notebook, the input images are readable from a location specified as an argument, and all output images and other requested results are displayed in the notebook environment. All cells in your notebook must have been executed so that the tutor/marker does not have to execute the notebook again to see the results.

Copyright: UNSW CSE COMP9517 Team. Reproducing, publishing, posting, distributing, or translating this lab assignment is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.

Released: 22 September 2026